Inside Python — How It Runs & Its Building Blocks

Class 2 — In-Class Work-Along Sheet (working title · built iteratively)

Name: _____ Section: _____ Date: _____

How to use this sheet: Fill it in *as we go*. Attempt every question **before** we discuss it — your first guesses do not need to be right; thinking hard and being wrong is exactly how this is meant to work. Keep your **Terminal** open for the hands-on bits.

Part 1 — What happens when you run `python file.py`?

Q1. You type `python hello_world.py` and press **Enter**. Who — or *what* — actually **runs** your file? Write your best guess (it's completely fine to be wrong).

_____

_____

Q2. *(After we discuss it — books closed.)* In your own words, what are the **two things** Python does when you run `python file.py`?

_____

_____

Q3. Python *could* try to run your text directly, but it first **translates** your code into something called **bytecode**. Give **one reason why** that extra translation step is worth it.

_____

Q4. *(Hands-on — like the Class-1 demo.)* Re-run the program that **imports another file**, then open the `__pycache__` folder. (a) Which file(s) got a `.pyc`?
_____ (b) Why does Python bother **saving** them? _____

Q5. *(Reflection.)* Name **one thing** about "how Python runs your code" you'd like cracked open in a later class.

_____

Part 2 — What did "installing Python" actually put on your computer?

Q6. *(Guess first.)* You "installed Python." Where on your computer do you think the `python` **program itself** lives? Take a guess at a folder/path.

_____

Q7. *(Now find it.)* Run these two commands and write what you get:

```
which python3
python3 -c "import sys; print(sys.executable)"
```

(a) The path it prints: _____

(b) Are the two outputs the same? _____ (c) In one line — what is this path pointing to?

_____

Q8. *(Open the box — the "standard library".)* Run this (it lists files that came bundled with your Python):

```
STDLIB=$(python3 -c 'import sysconfig; print(sysconfig.get_path("stdlib"))')
ls "$STDLIB" | grep -E "random|json|datetime|statistics|csv|turtle"
```

Write **3 module names you recognize:** _____

What does seeing these *files* tell you about what really happens when you write `import random`?

Q9. *(Built-in vs "you install it".)* Run these two lines:

```
python3 -c "import math; print('works:', math.factorial(5))"
python3 -c "import emoji"
```

(a) One works instantly, the other fails — **why?**

(b) What **single command** would fix the failing one? *(Hint: it starts with `python3 -m pip ...`)*

Q10. *(Use a built-in module yourself.)* First run this little program:

```
import random
print("Your lucky number:", random.randint(1, 100))
```

Now write your **own 2 lines** that `import` a **different** built-in module and do something with it:

```
+-------------------------------------------------------------------------+
|                                                                         |
|                                                                         |
+-------------------------------------------------------------------------+
```

Q11. *(Your install + a surprise.)* How did **you** get Python — from a **website** (downloaded an installer and double-clicked) or from the **command line** (a package manager like `brew` / `conda`)? _____

Now run:

```
which -a python3
```

How many *different* Pythons are on your machine? _____ Surprised?

Q12. *(The classic trap.)* Each Python installation has its **own pip**. If you just type `pip install something`, **which** Python actually gets the package? And what **safer command** guarantees it lands in the same Python you run with `python3`?

Q13. *(Recall — books closed.)* List the parts that together make up "Python software."

TAKE-HOME ACTIVITY (do later) — "Two Ways to Install, Then X-Ray Both"

You learned to X-ray a Python install. Now install **two** small programs — **one the *website* way and one the *command-line* way** — then compare how they arrive and how they're built.

① **Install one program from a WEBSITE** (the download-and-double-click way). Pick something small from its **official site** (e.g. a small utility app — 7-Zip, VLC, Rectangle, etc.) and run the installer.

② **Install one program from the COMMAND LINE** (using a *package manager*). Good tiny choices: `tree`, `cowsay`, or `jq`. *Don't have a package manager yet? Install one first:*

```
Windows : you likely already have winget  ->  check:  winget --version
          (if missing, install "App Installer" from the Microsoft Store)
          then:   winget install <tool>
macOS   : install Homebrew — paste this in Terminal:
          /bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
          then:   brew install <tool>
Linux   : you already have apt  ->  sudo apt update && sudo apt install <tool>
```

③ **X-ray BOTH.** For each one find: *Where did it land?* (`which <name>` for the command-line tool) · *Is it one single file, or a bundle of many? · Which file actually runs?*

④ **Compare — write 6–8 sentences:** Which method felt easier? Where did each put its files? Was one a single binary and the other a bundle? If you had to install **20** tools, which way would you choose — and why?

✅ You **may use AI tools (Claude / ChatGPT)** as a helper (e.g. paste an `ls` and ask *"what is each of these files for?"*). **Bring your findings to share next class.**

Part 3 — Python Versions

In Part 2 you found your machine has **several Pythons** (e.g. `3.13`, `3.12`, `3.11`, `3.9`). Those aren't copies — they're different **versions**. Here's what that means.

Q14. Run `python3 --version`. (a) Your version: _____ (b) Python versions look like `MAJOR.MINOR.PATCH`. Label yours: MAJOR = _____, MINOR = _____, PATCH = _____ (c) A **patch** bump (last number) brings only bug fixes. What kind of change can a **major** bump (first number) bring?

Q15. *(Predict, then watch.)* The `match` statement was **added in Python 3.10**. Here is a small program:

```
x = 5
match x:
    case 5:
        print("match works!")
    case _:
        print("something else")
```

Predict — will it run on **Python 3.13**? _____ On **Python 3.9**? _____ Now watch it run on both. **What happened, and why?**

Q16. *(The rule + a key term.)* Fill in **works** or **breaks**:

- Old code (written for 3.9) run on a **newer** Python (3.13) → usually _____.
- New code (using a 3.13 feature) run on an **older** Python (3.9) → usually _____.

The property of a *newer* version still being able to run *older* code is called:

_____

In one line — **why** the asymmetry?

Q17. A program written in **2008** (Python **2**) often will **not** run on today's Python 3. What kind of version change explains that? *(major / minor / patch?)* _____

Q18. *(Reflection.)* You'll soon use AI libraries (PyTorch, TensorFlow…) that are **very** picky about Python versions. Name **one way** a version mismatch could bite you.

Part 4 — IDEs, "Run" Buttons & Online IDEs

Q19. *(Guess first.)* When you click the **Run** button in your code editor, what do you think happens *under the hood*? Is it any different from typing `python file.py` in the terminal?

Q20. *(Observe.)* In your editor (e.g. VS Code) click **Run**, then look at its **integrated terminal**. Write the **actual command** it ran:

```
+-------------------------------------------------------------------------+
|                                                                         |
+-------------------------------------------------------------------------+
```

Is it different from just typing `python file.py` yourself?

Q21. *(What went wrong? — think BEFORE the reveal.)* Your instructor writes a small program that uses a library (PyYAML), then:

- runs it in the **Terminal** → it prints the answer, no problem;
- clicks **Run** on the **same file in the IDE** → it crashes with `ModuleNotFoundError: No module named 'yaml'`.

Same file, same laptop — why does it work in the Terminal but not in the IDE?

(After the reveal) In one line — what should you check/fix **in the IDE**? *(Hint: recall Q20 + Part 2 — which* _____ *is the IDE's Run button using?)*

Q22. *(Online IDEs.)* When you run code on an online IDE like **Google Colab** or **Replit**, **whose computer** actually runs your code — yours, or someone else's? Briefly trace how your code gets there and the result comes back.

Q23. *(Unify — recall.)* In **one sentence**: what is common to (a) typing `python file.py`, (b) clicking **Run** in VS Code, and (c) hitting **Run** in Colab?

Q24. *(AI reflection.)* Why is **Google Colab** especially useful for AI / ML work?

Part 5 — Names, Values & the Assignment Operator

Q25. *(Guess first — your own words.)* Almost every language lets you attach a **value** to a **variable**. So there are two ideas: a **variable** and a **constant**. How are they different? *(A tiny example helps.)*

Q26. Look at `x = 7`. (a) What is `x`? *(Python has its own word for it.)* _____ (b) What does the `=` do — and is it the **same** as "equals" in maths?

Q27. *(The tricky one — predict.)* If `x` is `6`, what does `x = x + 1` make `x`? **Explain why.** *(Remember: `=` is assignment, not equality.)*

Q28. *(Statements.)* A **statement** is one complete instruction Python runs — normally **one per line**. Write `x = 7` and `y = 9` as **two statements on a single line**, in **two different ways**:

```
+-------------------------------------------------------------------------+
|  way 1:                                                                 |
|                                                                         |
|  way 2:                                                                 |
+-------------------------------------------------------------------------+
```

Q29. *(Predict-then-run.)* (a) Do `x=7`, `x = 7`, and `x = 7` all behave the **same**? _____ (b) Now put `y = 9` on the next line with a **tab** in front of it, then run it. What happens, and **why**?

Q30. *(Recall — books closed.)* Fill in:

- Python calls a variable a _____.
- The `=` symbol is the _____ operator (not the equality operator).
- "Spacing around `=` is free, but _____ across lines is part of Python's syntax."

Part 6 — Data Types

Q31. *(Guess.)* Last time we wrote `x = 7`. That `7` was a particular **type** of data. **What type do you think it is?**

Q32. *(Discover with `type()`.)* `type` is a **built-in function** — it came *with* Python (you never defined it). Run:

```
print(type(7))
print(type(3.14))
print(type(True))
print(type("Hello Zenith"))
```

Write what each line prints:

Q33. *(The char question.)* In Java / C++ a single letter is a `char`. Run:

```
print(type('a'))
print(type('apple'))
print(len('a'))
```

(a) Are `'a'` and `'apple'` the **same** type? _____ (b) So does Python have a separate **character** type? What *is* `'a'`, then?

Q34. *(Predict — the big one.)* What happens when you run this — will it **crash** or **work**? And **why**?

```
x = 7
x = "I love my India"
print(x)
```

Q35. *(Contrast.)* Here is the *same idea* in **C++**:

```
int x = 1;
x = 3;                  // line A
x = "I love my India";  // line B
```

Which line would C++ **accept**, and which would it **reject**? Why is C++ different from Python here?

Q36. *(Terms — recall.)*

- Languages where a name can hold (and switch to) **any** type, like Python, are called _____ typed.
- Languages where a variable's type is **fixed** at declaration, like C++/Java, are _____ typed.
- Python is _____ typed.

Q37 — Mini-research (use AI!). Ask an AI tool (Claude / ChatGPT): *"What are the trade-offs between dynamically typed and statically typed languages?"* Then, in your own words, write **2 advantages of each**:

- **Dynamic** (e.g. Python): _____
- **Static** (e.g. C++ / Java): _____

Part 7 — Operators: Doing Things With Data

Q38. *(Guess.)* A name just **stores** data — but a program needs to **do** things with it. What kinds of **operations** do you do with data? And **where** on the computer do these operations actually happen?

Q39. *(Fill in.)* Operations are carried out by the _____ (the computer's "calculator"). Modern PCs have **multiple** _____ _____, so they can do many operations at once. So a program's **speed** depends partly on the _____.

Q40. *(Operators & operands.)* In `7 + 3`, the `+` is the **operator** and `7`, `3` are its **operands**. Compute:

```
print(7 + 3)
print(7 - 3)
print(7 * 3)
```

Answers: _____ , _____ , _____

Q41. *(Predict-then-run.)* Predict the output, then run:

```
print(6 / 2)
print(7 / 3)
```

(a) `6 / 2` → _____ (b) `7 / 3` → _____

What's **surprising** about `6 / 2`? *(Hint: what **type** is the result?)*

Q42. *(Recall — books closed.)*

- An operator that works on **two** values is called a _____ operator; the values it works on are its _____.
- In Python, the `/` operator always returns a _____ (this is called **true division**).

Part 8 — Converting Between Types

Implicit (automatic) conversion — mixed types get "converted up"

Q43. *(Predict — value AND type.)* Predict the output **and the type** of each, then run:

```
print(7 + 3)
print(7 + 3.0)
print(7 / 2)
```

Write value → type for each:

Q44. *(The rule.)* When an `int` and a `float` meet in an operation, what **type** is the result? **Why?**

Q45. *(The ladder.)* Order these numeric types from **simplest** to **most complex**: `float`, `int`, `complex`.

Q46 — Mini-research (use AI!). We haven't taught **complex numbers** yet — go learn the type. Write a tiny program that **creates** a complex number (e.g. `2 + 3j`) and does **one operation** on it. Also: what **type** is `7 + 2j`?

```
+-------------------------------------------------------------------------+
|                                                                         |
|                                                                         |
+-------------------------------------------------------------------------+
```

Explicit (manual) conversion — you convert it yourself

Q47. *(Predict, then run.)* `int()`, `float()`, `str()` are **built-in functions** that convert a value's type. Predict each, then run:

```
print(float(7))
print(int(3.9))
print(str(7))
```

(a) `float(7)` → _____ (b) `int(3.9)` → _____ — does it **round** or **chop**? (c) `str(7)` → _____

Q48. *(Strings → numbers — the probes.)* Predict each, then run:

```
print(int("42"))
print(int("256tydv7w"))
```

(a) `int("42")` → _____ (b) What happens with `int("256tydv7w")`? **Why?**

Q49. *(Recall — books closed.)*

- When Python converts a type **automatically** during an operation, it's called _____ conversion.
- When **you** convert it yourself with `int()` / `float()` / `str()`, it's called _____ conversion.